

4. VARIABLES METHODS

Data types can be categorized into two types

1. Primitive
2. Non-Primitive

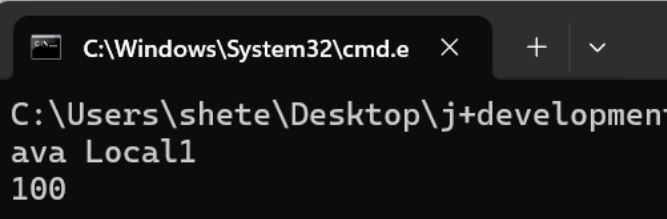
Primitive and non-primitive can be further be categorized into three types

- Local
- Static
- Non-Static

Local Variables or local datatypes

Variables which are declared within a method or constructor or blocks such variables are called as local variables.

```
class Local1
{
    public static void main (String [] args)
    {
        //local variable
        int a =100;
        System.out.println(a);
    }
}
```



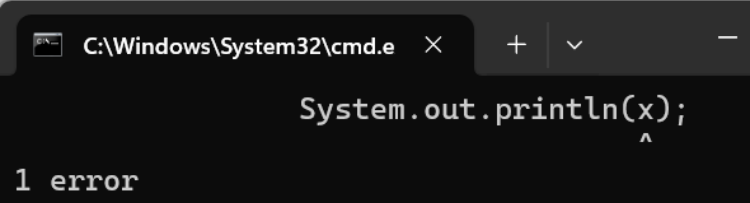
The screenshot shows a terminal window with the command `java Local1` and the output `100`. The terminal title is `C:\Windows\System32\cmd.e`.

IMP note: We can access local variables directly

Is it possible to print local variable without initializing?

Ans: it is not possible

```
class Local2
{
    public static void main (String [] args)
    {
        int x;
        System.out.println(x);
    }
}
```

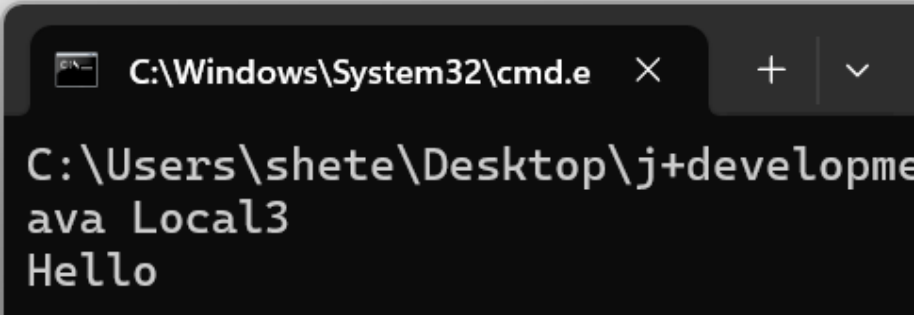


The screenshot shows a terminal window with the command `java Local2` and the output `1 error`. The terminal title is `C:\Windows\System32\cmd.e`. The error message is `System.out.println(x);` with an arrow pointing to the variable `x`.

4. VARIABLES METHODS

It is possible in java to just declare variable without initializing.

```
class Local3
{
    public static void main (String [] args)
    {
        int z;
        System.out.println("Hello");
    }
}
```



C:\Windows\System32\cmd.e

C:\Users\shete\Desktop\j+development>java Local3

Hello

4. VARIABLES METHODS

Static Variable

Variables which are declared outside the method or constructor or block but within the class and is prefix with static keyword such keyword called as static variables.

How do we access static keyword?

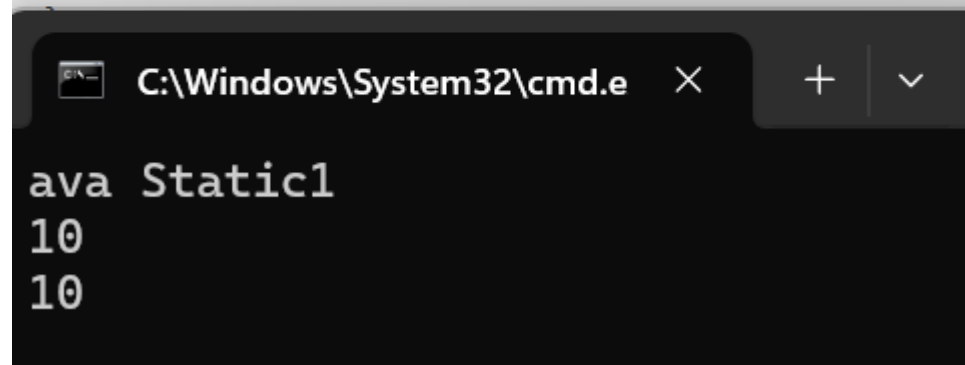
Ans: **. operator or access operator** to access a member within a specific area we use **.** operator or also called as access operator

How many ways are there to access to static variables?

Ans: There are three approaches

1. Directly
2. Class Name
3. Object

```
class Static1
{
    static int z=10;
    public static void main (String [] args)
    {
        //Directly
        System.out.println(z);
        //class name
        System.out.println(Static1.z);
    }
}
```

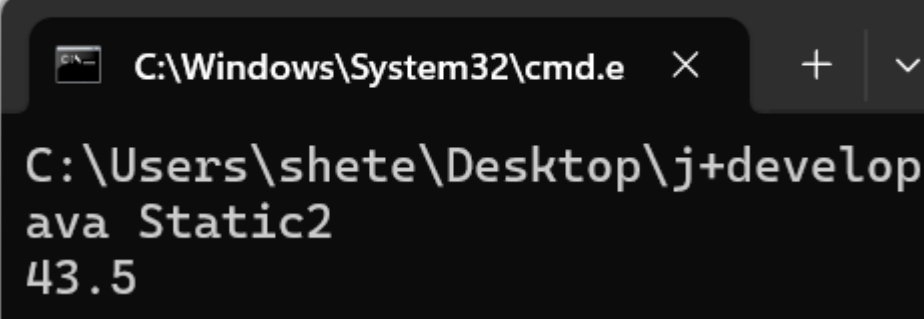


```
C:\Windows\System32\cmd.e
java Static1
10
10
```

4. VARIABLES METHODS

Write a java program to add two static variables?

```
class Static2
{
    static int x=10;
    static float a=33.5f;
    public static void main (String [] args)
    {
        //Directly
        float result;
        result=x+a;
        System.out.println(result);
    }
}
```

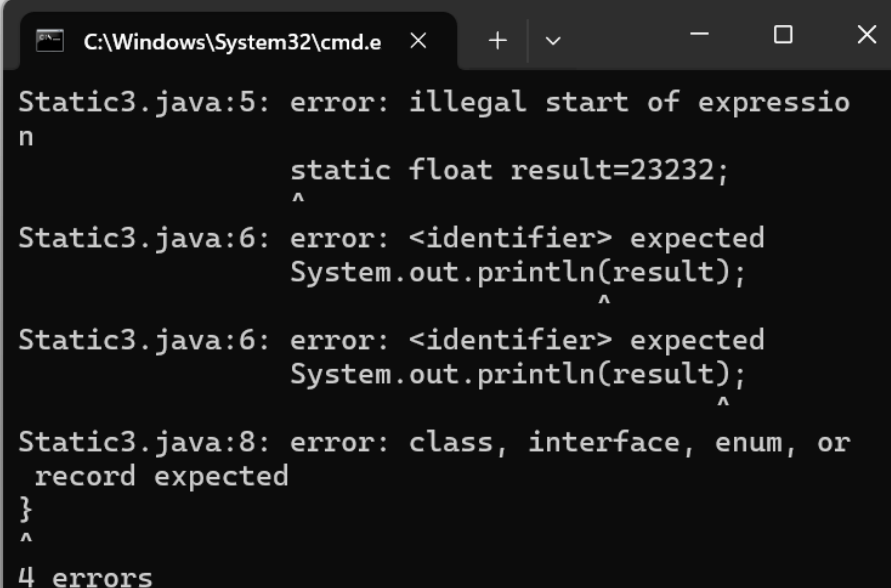


C:\Windows\System32\cmd.e X + v

C:\Users\shete\Desktop\j+develop
ava Static2
43.5

IMP note: we cannot use static keyword for local variables.

```
class Static3
{
    public static void main (String [] args)
    {
        static float result=23232;
        System.out.println(result);
    }
}
```



C:\Windows\System32\cmd.e X + v - □ X

Static3.java:5: error: illegal start of expression
n
static float result=23232;
^

Static3.java:6: error: <identifier> expected
System.out.println(result);
^

Static3.java:6: error: <identifier> expected
System.out.println(result);
^

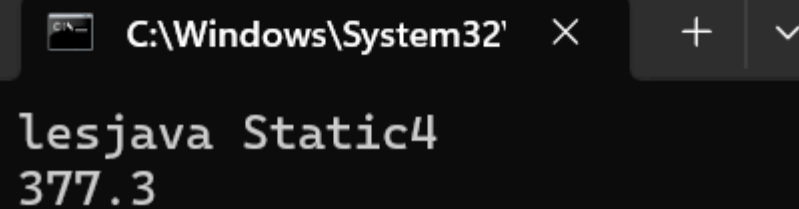
Static3.java:8: error: class, interface, enum, or
record expected
}
^

4 errors

4. VARIABLES METHODS

```
class Static4
{
    static int var=34;
    static float result;

    public static void main (String [] args)
    {
        float var2=343.3f;
        result= var+var2;
        System.out.println(result);
    }
}
```



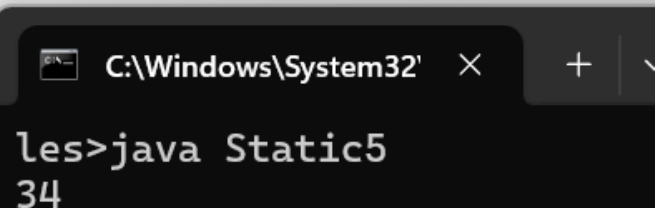
C:\Windows\System32' X + v

```
lesjava Static4
377.3
```

How many ways are there to access static variables from another class?

1. Class name
2. Object

```
class Demo
{
    static int z=34;
}
class Static5
{
    public static void main (String [] args)
    {
        System.out.println(Demo.z);
    }
}
```



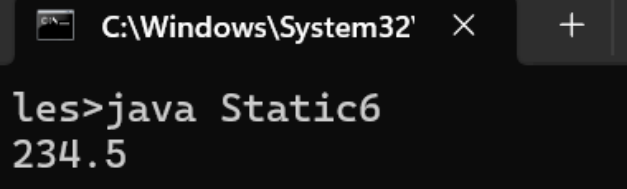
C:\Windows\System32' X + v

```
les>java Static5
34
```

4. VARIABLES METHODS

```
class Demo
{
    static int z=34;
}
class Sample
{
    static float var=200.5f;
}

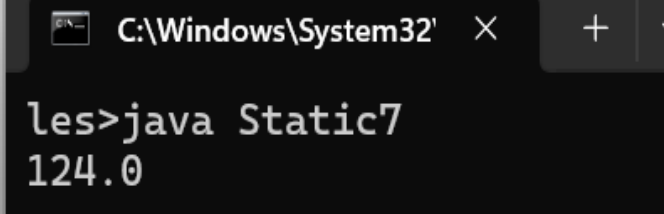
class Static6
{
    public static void main (String [] args)
    {
        float result;
        result=Demo.z+Sample.var;
        System.out.println(result);
    }
}
```



C:\Windows\System32' X +

les>java Static6
234.5

```
class Demo
{
    static float result;
}
class Sample
{
    static int var1=24;
}
class Static7
{
    public static void main (String [] args)
    {
        int var2=100;
        Demo.result=Sample.var1+var2;
        System.out.println(Demo.result);
    }
}
```

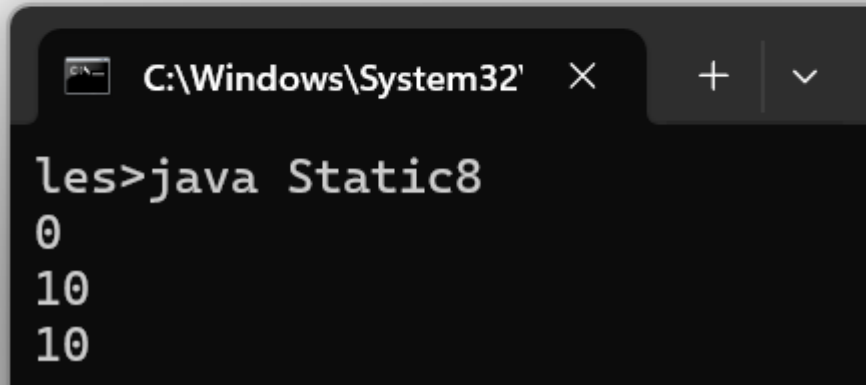


C:\Windows\System32' X +

les>java Static7
124.0

4. VARIABLES METHODS

```
class Static8
{
    static int a;
    public static void main (String [] args)
    {
        System.out.println(a);
        a=10;
        System.out.println(a);
        System.out.println(Static8.a);
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command 'les>java Static8' has been entered and executed. The output of the program is displayed on three separate lines: '0', '10', and '10'.

```
C:\Windows\System32>les>java Static8
0
10
10
```

4. VARIABLES METHODS

Methods

Methods in java can be categorized into two types

- 1.static
2. Non-static

Method can be defined as block of code which are meant to perform specific operation.

What is benefit of using Method.

The benefit of using methods are:

- a) It will help to reduce or avoid duplicate code.
- b) It will help reduce the code length
- c) It will help consume user memory.
- d) It will consume less time for execution.
- e) The application speed will be faster.

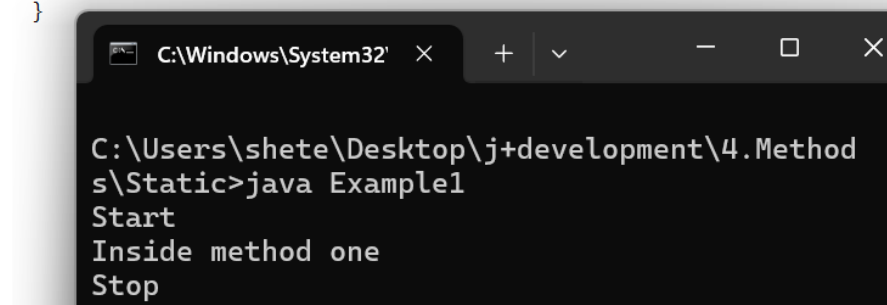
Syntax:

[Access modifier] [modifier] return type method name (variables arguments)

```
{  
    Statement;  
    Return  
}
```

1.Return statement is used to return the control back to the caller method.

```
class Example1  
{  
    public static void main (String [] args)  
    {  
        System.out.println("Start");  
        method1();  
        System.out.println("Stop");  
    }  
  
    static void method1()  
    {  
        System.out.println("Inside method one");  
        return;  
    }  
}
```



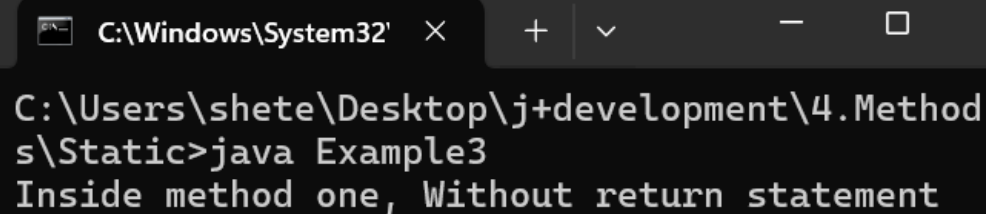
```
C:\Windows\System32' x + - □ x  
C:\Users\shete\Desktop\j+development\4.Method  
s\Static>java Example1  
Start  
Inside method one  
Stop
```


4. VARIABLES METHODS

2.If we do not use return statement the control will be lost and other statement will not get executed.

If we do not use return statement then the compiler will set the return statement by default.

```
class Example3
{
    public static void main (String [] args)
    {
        method1();
    }
    static void method1()
    {
        System.out.println("Inside method one, Without return statement ");
    }
}
```

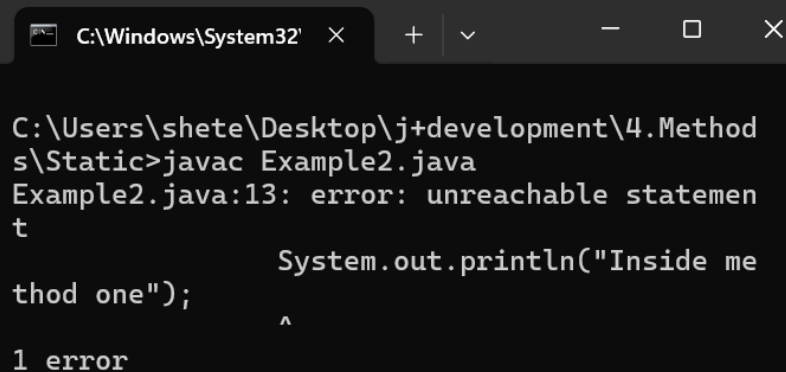


```
C:\Windows\System32' x + v - □ x
C:\Users\shete\Desktop\j+development\4.Method
s\Static>java Example3
Inside method one, Without return statement
```

3.The return statement always should be used at end of method.

```
class Example2
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        method1();
        System.out.println("Stop");
    }

    static void method1()
    {
        return;
        System.out.println("Inside method one");
    }
}
```



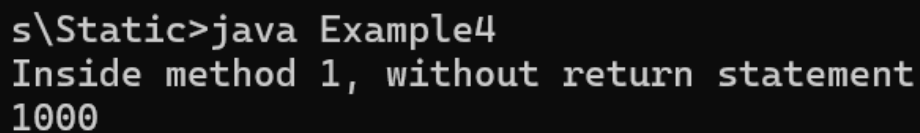
```
C:\Windows\System32' x + v - □ x
C:\Users\shete\Desktop\j+development\4.Method
s\Static>javac Example2.java
Example2.java:13: error: unreachable statemen
t
        System.out.println("Inside me
thod one");
        ^
1 error
```

4. VARIABLES METHODS

4. When should we go for pass by value

Whenever we want to access local variables, we go for pass by value.

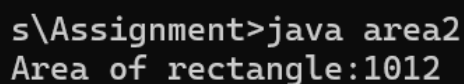
```
class Example4
{
    public static void main (String [] args)
    {
        int a= 1000;
        method1(a); //pass by value
    }
    static void method1(int a)
    {
        System.out.println("Inside method 1, without return statement");
        System.out.println(a);
    }
}
```



```
C:\Windows\System32' x + v - □ ×
s\Static>java Example4
Inside method 1, without return statement
1000
```

Q1. Write a java program to display the area of triangle

```
class area2
{
    public static void main(String[] args)
    {
        int a=23,b=44;
        rectangle(a,b);
    }
    static void rectangle(int a,int b){
        int area=a*b;
        System.out.println("Area of rectangle:"+area);
        return;
    }
}
```

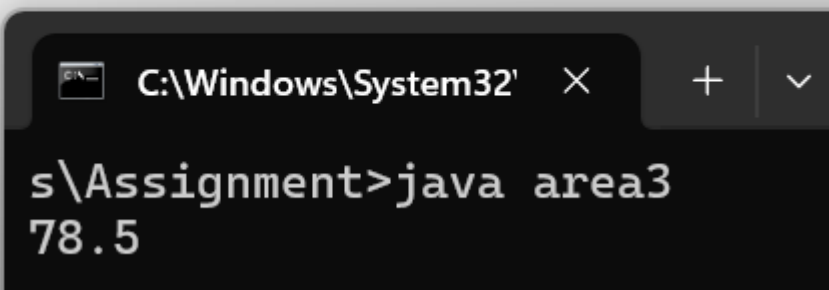


```
C:\Windows\System32' x + v
s\Assignment>java area2
Area of rectangle:1012
```

4. VARIABLES METHODS

Q2. Write a java program to calculate area of circle.

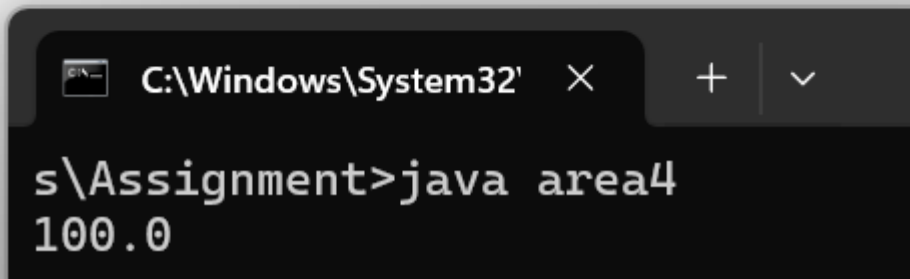
```
class area3
{
    public static void main(String[] args)
    {
        double r=5;
        area(r);
    }
    static void area(double x){
        double a=3.14*x*x;
        System.out.println(a);
    }
}
```



```
C:\Windows\System32' x + v
s\Assignment>java area3
78.5
```

Q3. Write a java program to find area of rectangle.

```
class area4
{
    public static void main(String[] args)
    {
        double a=10,b=20;
        area(a,b);
    }
    static void area(double b,double h){
        double a=0.5*b*h;
        System.out.println(a);
    }
}
```



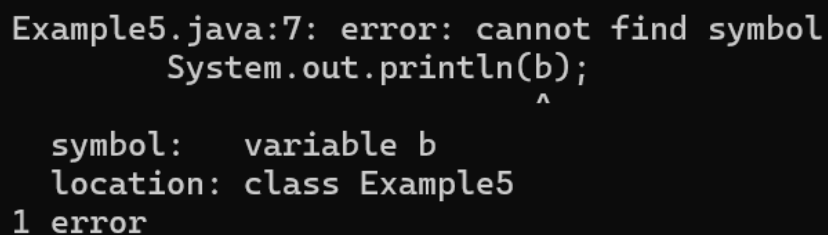
```
C:\Windows\System32' x + v
s\Assignment>java area4
100.0
```

4. VARIABLES METHODS

5.If a method is associated with the void keyword when the method has to only return a control and nothing else.

6.Depending on the data to be returned the method return type should be of same type.

```
class Example5
{
    //caller
    public static void main(String [] args)
    {
        float a= test1();
        System.out.println(b);
    }
    //called
    static float test1()
    {
        System.out.println("Inside test1");
        float a=10.5f;
        return a;
        //return by value
    }
}
```



```
Example5.java:7: error: cannot find symbol
    System.out.println(b);
                      ^
symbol:   variable b
location: class Example5
1 error
```

7.When we should go for return by value?

- Whenever we are moving control from caller method to called method we have to go for pass by value.
- Whenever we have to move the control from called method to caller value we have to go for return by value.
- Whenever we have to pass multiple values to the method a method can accept multiple values.
- Hence in pass by value we can pass multiple values.
- But whenever a method has to return a value, it can return only one value at any point of time.
- A method can return only one value because a method will have only one value.

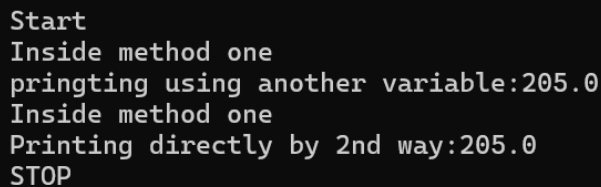
4. VARIABLES METHODS

```
class Example6
{
    public static void main (String [] args)
    {
        System.out.println("Start");
        int a=10;
        float b=20.5f;

        //there is two ways to print the result
        //first by declaring another variable 'result'
        float result=multiply(a,b);
        System.out.println("pringting using another variable:"+result);

        //second directly
        System.out.println("Printing directly by 2nd way:"+multiply(a,b));

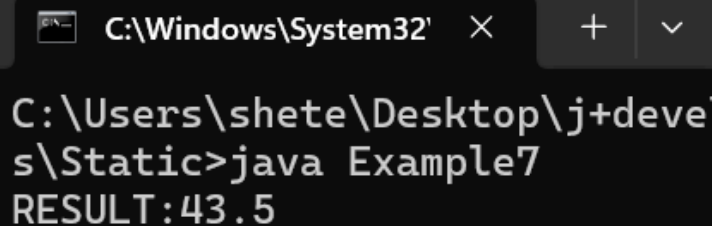
        System.out.println("STOP");
    }
    static float multiply(int a, float b)
    {
        System.out.println("Inside method one");
        float result= a*b;
        return result;
    }
}
```



Start
Inside method one
pringting using another variable:205.0
Inside method one
Printing directly by 2nd way:205.0
STOP

```
class Sample
{
    static float m1(int a, float b)
    {
        float result=a+b;
        return result;
    }
}
class Example7
{
    public static void main (String [] args)
    {
        int a= 33;
        float b=10.5f;

        float result=Sample.m1(a,b);
        System.out.println("RESULT:"+result);
    }
}
```

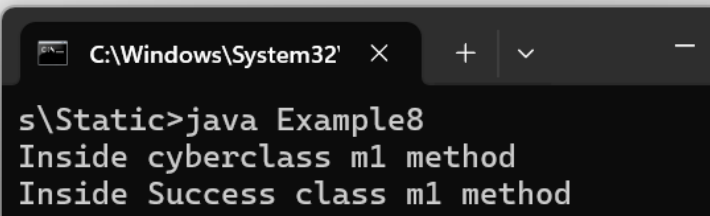


C:\Windows\System32' X + v
C:\Users\shete\Desktop\j+deve
s\Static>java Example7
RESULT:43.5

4. VARIABLES METHODS

Variables which are declared with in main or part of method or constructors or blocks, such variables are called as local variables.

```
class Cyber
{
    static void m1()
    {
        System.out.println("Inside cyberclass m1 method");
    }
}
class Success
{
    static void m2()
    {
        System.out.println("Inside Success class m1 method");
    }
}
class Example8
{
    public static void main (String [] args)
    {
        Cyber.m1();
        Success.m2();
    }
}
```

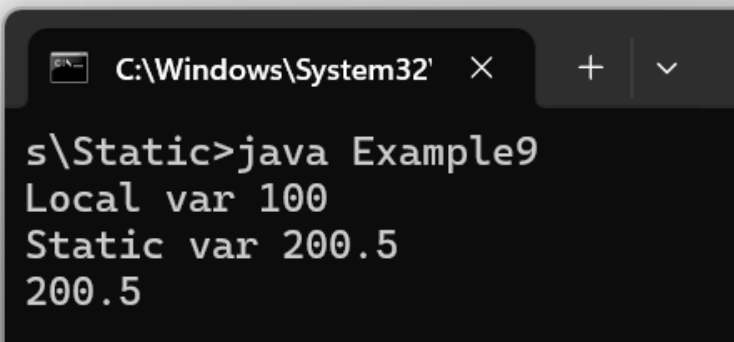


```
C:\Windows\System32' x + v -
s\Static>java Example8
Inside cyberclass m1 method
Inside Success class m1 method
```

Whenever JVM is called for execution, it will always and always consume RAM memory.

```
class Example9
{
    static float b=200.5f;
    public static void main (String [] args)
    {
        int a=100;
        //local variable
        System.out.println("Local var "+a);

        System.out.println("Static var "+b);
        System.out.println(Example9.b);
    }
}
```

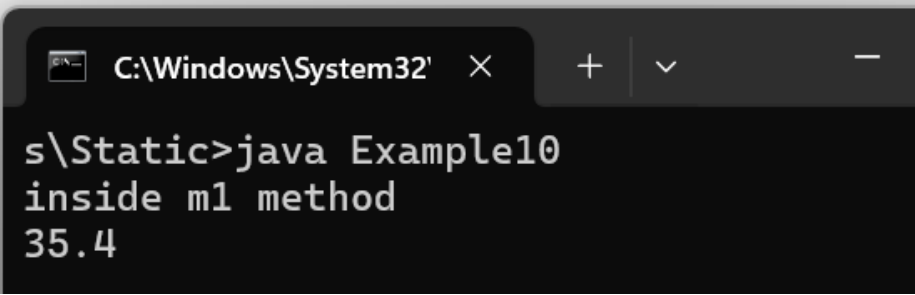


```
C:\Windows\System32' x + v -
s\Static>java Example9
Local var 100
Static var 200.5
200.5
```

4. VARIABLES METHODS

- Static members are always loaded within the static pool.
- Static members are loaded at class loading time.
- Main and any other methods are always executed in static area.
- Local variables are always created in stack area.
- Whenever a method look for variable the preference is always given to stack area.

```
class Example10
{
    public static void main (String [] args)
    {
        int a=23;
        float b=12.4f;
        float result=m1(a,b);
        System.out.println(result);
    }
    static float m1(int a,float b)
    {
        System.out.println("inside m1 method");
        float r= a+b;
        return r;
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command prompt shows the following text:

```
s\Static>java Example10
inside m1 method
35.4
```

Why Is the main method named as static?

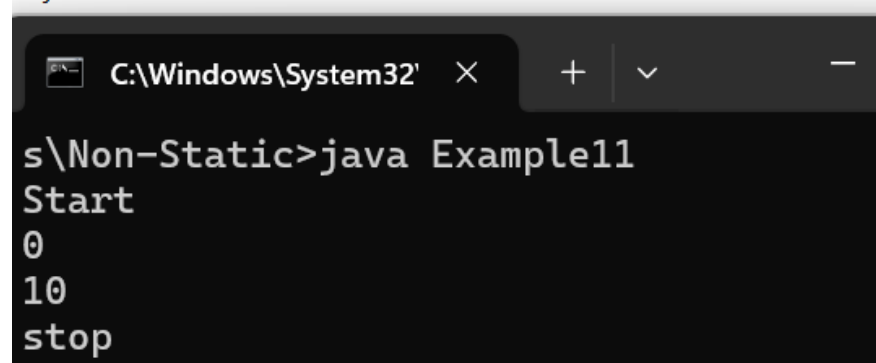
Whenever JVM enters the stack the first thing what the JVM does it will go to the static pool and look for main if the main is present then it will start the execution if the main is not present then execution can't be started.

4. VARIABLES METHODS

If a static variable is declared outside the method but initialize within a method such variables will still be considered as static variable.

```
class Example11
{
    static int a;
    public static void main(String [] args){
        System.out.println("Start");
        System.out.println(a);
        a=10;
        System.out.println(a);

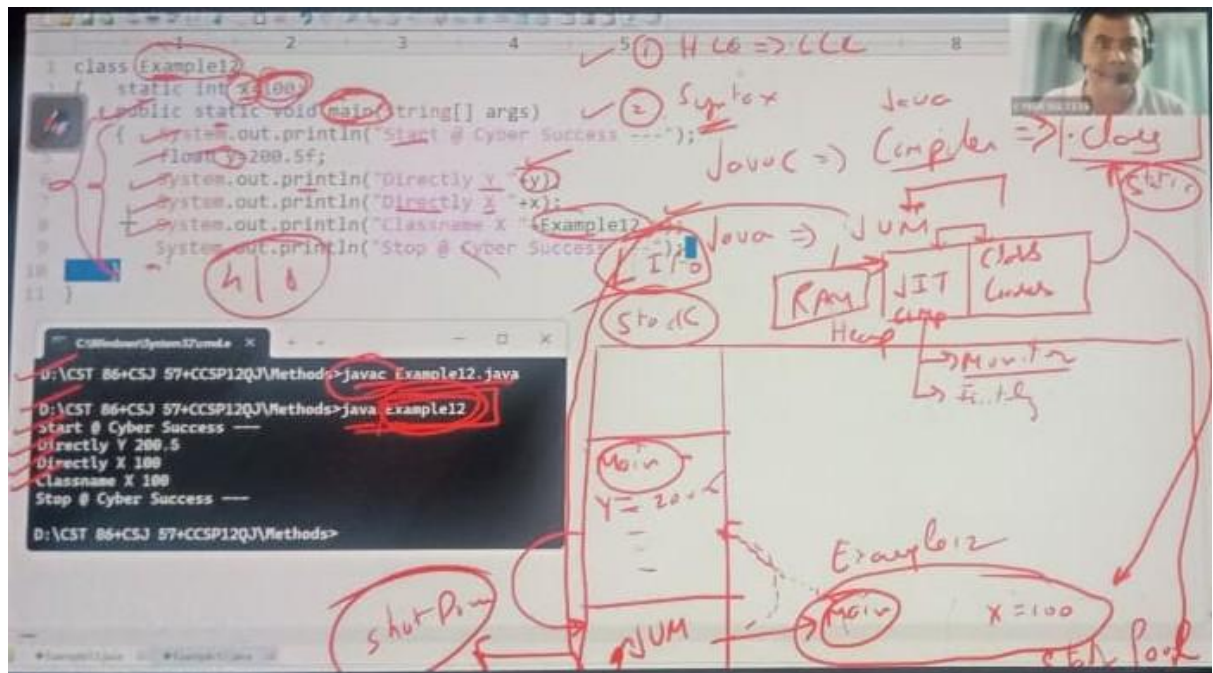
        System.out.println("stop");
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command prompt displays the following output:

```
s\Non-Static>java Example11
Start
0
10
stop
```


4. VARIABLES METHODS



1. Whenever JVM comes for execution, The First thing it performs is it splits the RAM into two parts one is Stack and One is Heap after splitting.
2. After splitting the controller is given to Class loader along with other components like JIT (Just in Time Compiler). The other components including JIT and class loaders are part of the JVM.
3. The Class loaders are responsible to load the static members within the static pool during the class loading the JIT compiler will not only monitor but will also initialize during this particular period.
4. The Static members whenever loaded is called as static pool as is given the same name as class name.
5. Once loading happens JVM control enters the stack area, the first thing what JVM finds is main if the main is present it will start the execution if not present it will not start the execution the main method for execution will be loaded within the stack.
6. When the method is loaded within the stack it will create local variables within stack area.
7. The Stack always follows LIFO (Last in First Out) passion.
8. Once the main method performs the operation the controller will be given back to JVM and the main method comes out of stack area by giving controller to JVM and based on execution JVM will shut down by coming out of stack area.

4. VARIABLES METHODS

Non-static variables and methods

Variable declared outside the method, constructor and block but within the class and it is not prefix with the static keywords such variables are called as non-static variables.

How do we access non static members

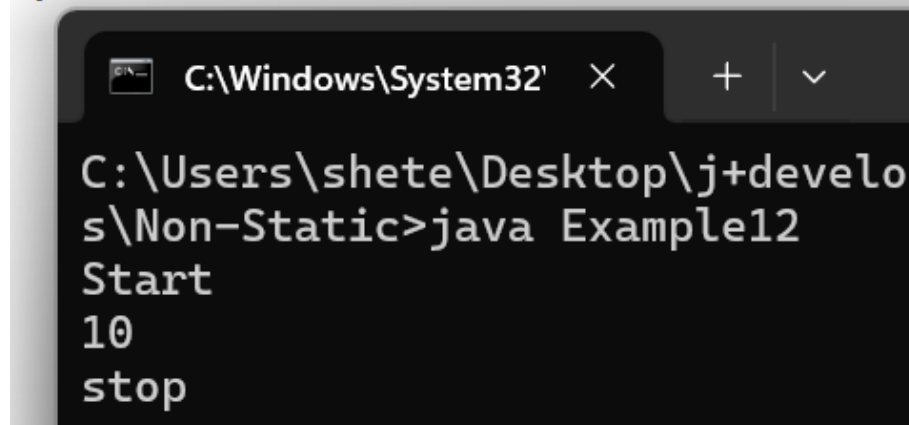
We can access non static members with the help of object creation.

Syntax for object creation.

Classname variable = new classname();

```
class Example12
{
    int a=10;
    public static void main(String [] args){
        System.out.println("Start");
        Example12 var = new Example12();
        System.out.println(var.a);

        System.out.println("stop");
    }
}
```



New

New is keyword

What exactly is role of new in object creation.

The new keyword is meant to perform three actions.

- It is made to create or allocated memory
- It is responsible to load non static members(methods,variables).
- It is made to help generate an address along with JVM.

4. VARIABLES METHODS

- Whenever an address of an object will generate it will be stored in a variable.
- If objects are created based on classes which means to create an object it is mandatory we required a class.

How many times will the object get created within program

Based on total no of new keywords as many times object will created

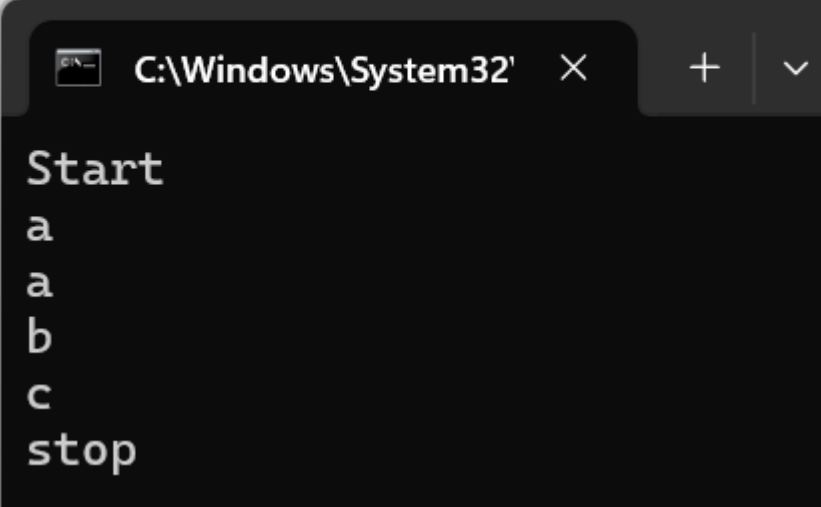
- Whenever there are changes made to one object it will never have an impact on another object because both object areas are different.

```
class Example13
{
    char x='a';
    public static void main(String [] args){
        System.out.println("Start");
        Example13 refvar1 = new Example13();
        Example13 refvar2 = new Example13();
        System.out.println(refvar1.x);
        System.out.println(refvar2.x);

        refvar1.x='b';
        refvar2.x='c';

        System.out.println(refvar1.x);
        System.out.println(refvar2.x);

        System.out.println("stop");
    }
}
```



C:\Windows\System32' × + ▾

Start
a
a
b
c
stop

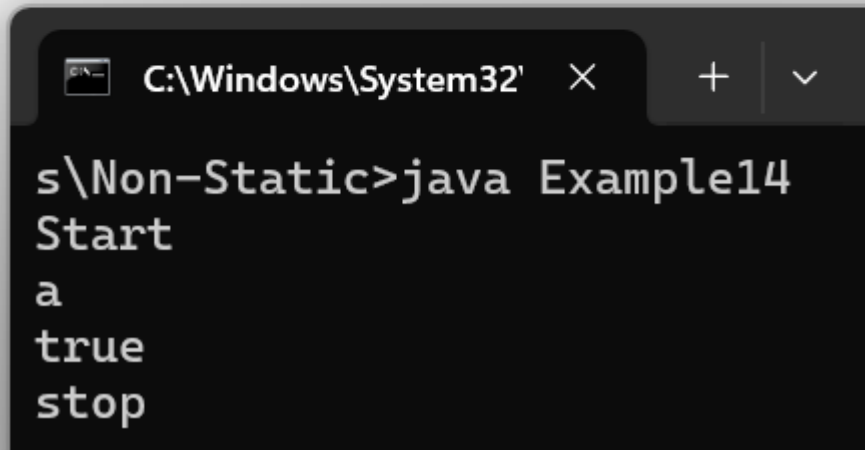
4. VARIABLES METHODS

A single object creation will be responsible to load all not static members of a respective class within the object

```
class Example14
{
    char x='a';
    boolean y=true;

    public static void main(String [] args){
        System.out.println("Start");

        Example14 ref=new Example14();
        System.out.println(ref.x);
        System.out.println(ref.y);
        System.out.println("stop");
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The command prompt displays the following output:

```
s\Non-Static>java Example14
Start
a
true
stop
```

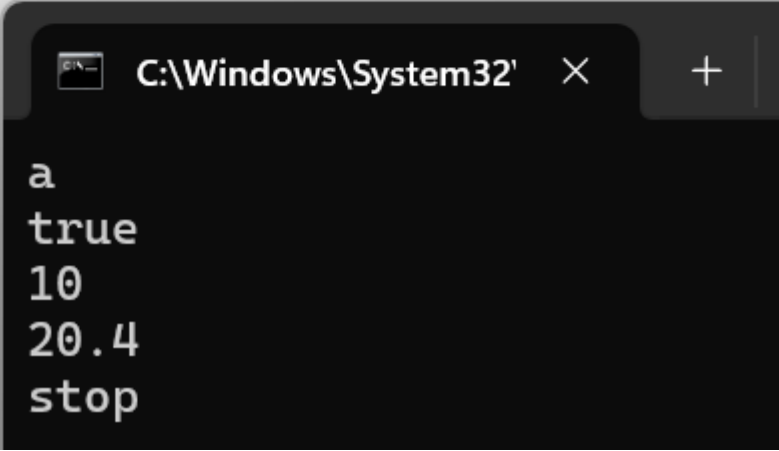
4. VARIABLES METHODS

```
class Cyber
{
    int a=10;
    float b=20.4f;
}
class Example15
{
    char x='a';
    boolean y=true;

    public static void main(String [] args){
        System.out.println("Start");

        Example15 ref=new Example15();
        System.out.println(ref.x);
        System.out.println(ref.y);

        Cyber c= new Cyber();
        System.out.println(c.a);
        System.out.println(c.b);
        System.out.println("stop");
    }
}
```



The screenshot shows a Windows command prompt window with the title bar 'C:\Windows\System32'. The output of the Java program is displayed as follows:

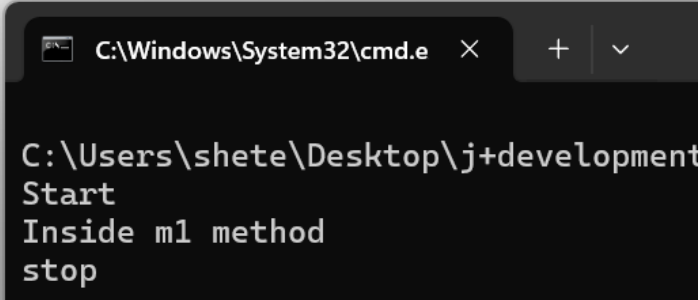
```
a
true
10
20.4
stop
```

4. VARIABLES METHODS

If a method is not prefix with keyword static then the static method becomes non-static.

```
class Example16
{
    public static void main(String [] args){
        System.out.println("Start");
        Example16 ref = new Example16();
        ref.m1();
        System.out.println("stop");
    }

    void m1(){
        System.out.println("Inside m1 method");
    }
}
```

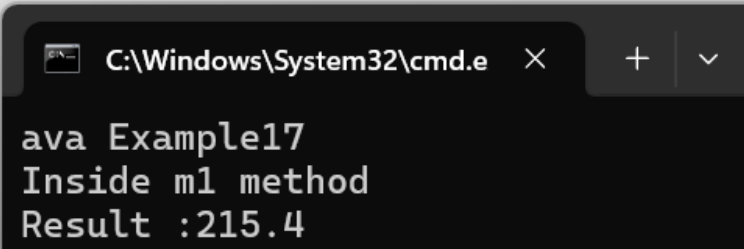


```
C:\Windows\System32\cmd.e  X  +  v

C:\Users\shete\Desktop\j+development
Start
Inside m1 method
stop
```

To access any local variable from static to non-static method or from non-static to static method we use pass by values or return by value.

```
class Example17
{
    public static void main (String [] args)
    {
        int a=10;
        float b=205.4f;
        Example17 ref = new Example17();
        float result= ref.m1(a,b);
        System.out.println("Result :"+result);
    }
    float m1(int a , float b)
    {
        System.out.println("Inside m1 method");
        float result=a+b;
        return result;
    }
}
```



```
C:\Windows\System32\cmd.e  X  +  v

ava Example17
Inside m1 method
Result :215.4
```

4. VARIABLES METHODS

///quadrant diagram

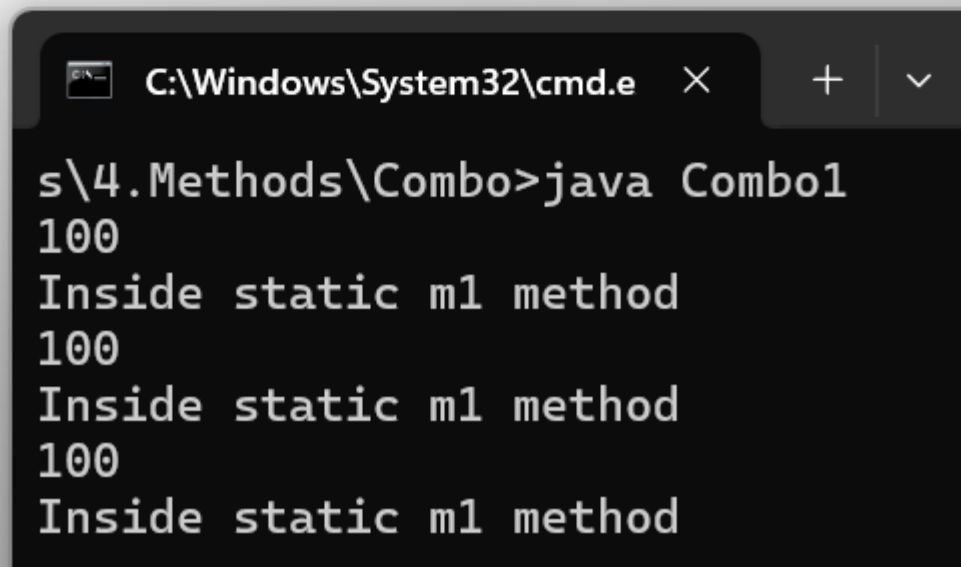
To be accessed	1	2	3	4
----------------	---	---	---	---

Members	Static	Static	Non static	Non static
Area	Static	Static	Static	Static
class	Single	Multiple	Single	Multiple

To be accessed	5	6	7	8
Members	Static	Static	Non static	Non static
Area	Non Static	Non static	Non Static	Non Static
class	Single	Multiple	Single	Multiple

4. VARIABLES METHODS

```
//static area
//static member
//single class
class Combo1
{
    static int a=100;
    static void m1()
    {
        System.out.println("Inside static m1 method");
    }
    public static void main (String [] args)
    {
        //dierctly
        System.out.println(a);
        m1();
        //classname
        System.out.println(Combo1.a);
        Combo1.m1();
        //object
        Combo1 ref= new Combo1();
        System.out.println(ref.a);
        ref.m1();
    }
}
```



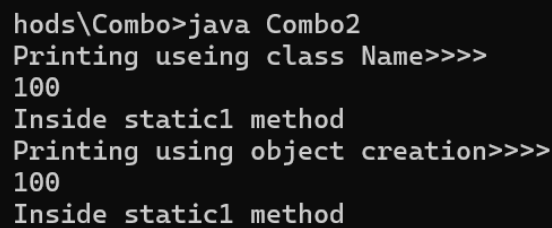
```
C:\Windows\System32\cmd.e  X  +  v

s\4.Methods\Combo>java Combo1
100
Inside static m1 method
100
Inside static m1 method
100
Inside static m1 method
```


4. VARIABLES METHODS

```
//static members
// static area
// multiple class
class Demo
{
    static int a=100;
    static void m1()
    {
        System.out.println("Inside static1 method");
    }
}
class Combo2
{
    public static void main (String [] args)
    {
        //class name
        System.out.println("Printing using class Name>>>>");
        System.out.println(Demo.a);
        Demo.m1();

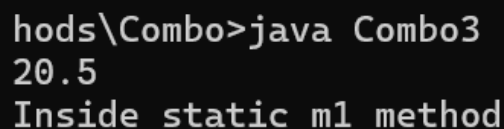
        //object
        Demo d = new Demo();
        System.out.println("Printing using object creation>>>>");
        System.out.println(d.a);
        d.m1();
    }
}
```



```
C:\Windows\System32\cmd.e  X  +  v

hods\Combo>java Combo2
Printing using class Name>>>>
100
Inside static1 method
Printing using object creation>>>>
100
Inside static1 method
```

```
//non static member
//static area
//single class
class Combo3
{
    float b=20.5f;
    static void m1()
    {
        System.out.println("Inside static m1 method");
    }
    public static void main (String [] args)
    {
        Combo3 c= new Combo3();
        System.out.println(c.b);
        c.m1();
    }
}
```



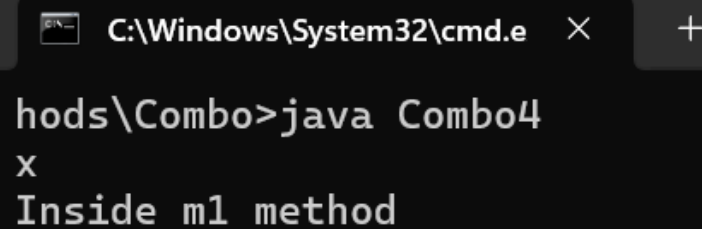
```
C:\Windows\System32\cmd.e  X  +  v

hods\Combo>java Combo3
20.5
Inside static m1 method
```

4. VARIABLES METHODS

```
//non static area
//static area
//multiple classes

class Sample
{
    char a='x';
    void m1()
    {
        System.out.println("Inside m1 method");
    }
}
class Combo4
{
    public static void main (String [] args)
    {
        Sample s= new Sample();
        System.out.println(s.a);
        s.m1();
    }
}
```

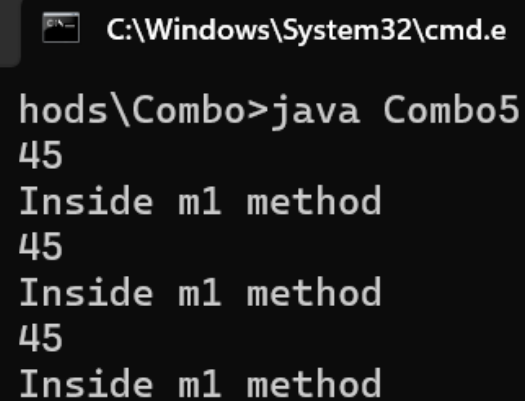


The screenshot shows a Windows command prompt window titled "C:\Windows\System32\cmd.e". The command prompt displays the output of the command "java Combo4". The output is "x" followed by "Inside m1 method" on the next line.

```
C:\Windows\System32\cmd.e  X  +
hods\Combo>java Combo4
x
Inside m1 method
```

4. VARIABLES METHODS

```
//static members
//non- static Area
//single class
class Combo5
{
    static int a=45;
    static void m1()
    {
        System.out.println("Inside m1 method");
    }
    public static void main (String[] args)
    {
        Combo5 ref= new Combo5();
        ref.test1();
    }
    void test1()
    {
        //directly
        System.out.println(a);
        m1();
        //class name
        System.out.println(Combo5.a);
        Combo5.m1();
        //object creation
        Combo5 c = new Combo5();
        System.out.println(c.a);
        c.m1();
    }
}
```

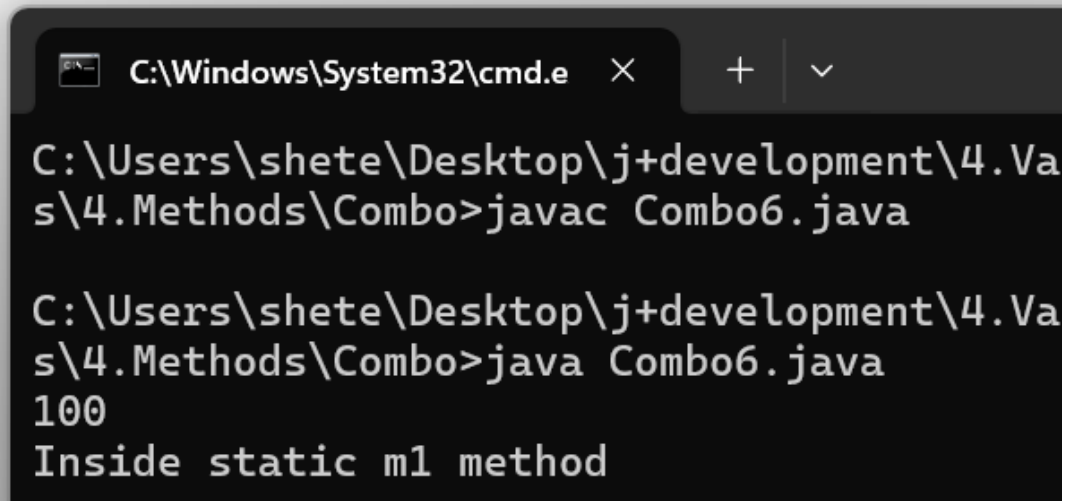


The screenshot shows a Windows command prompt window titled "C:\Windows\System32\cmd.e". The user has entered the command "hods\Combo>java Combo5". The output of the program is displayed as follows:

```
45
Inside m1 method
45
Inside m1 method
45
Inside m1 method
```

4. VARIABLES METHODS

```
//static members
//non- static Area
//multiple classes
class Cyber
{
    static int a=100;
    static void m1()
    {
        System.out.println("Inside static m1 method");
    }
}
class Combo6
{
    public static void main (String[] args)
    {
        Combo6 ref= new Combo6();
        ref.test1();
    }
    void test1()
    {
        //object creation
        Cyber c = new Cyber();
        System.out.println(c.a);
        c.m1();
    }
}
```



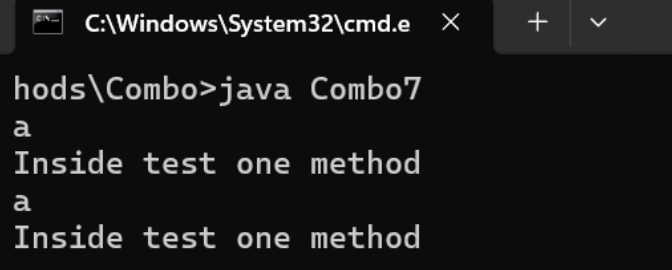
The screenshot shows a Windows command prompt window with the title bar "C:\Windows\System32\cmd.e". The window contains the following text:

```
C:\Users\shete\Desktop\j+development\4.Vas\4.Methods\Combo>javac Combo6.java

C:\Users\shete\Desktop\j+development\4.Vas\4.Methods\Combo>java Combo6.java
100
Inside static m1 method
```

4. VARIABLES METHODS

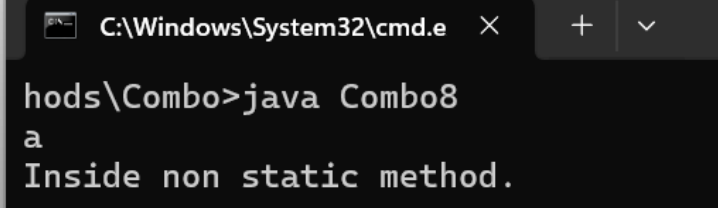
```
//non static member
//non static area
//single class
class Combo7
{
    char x='a';
    void test1()
    {
        System.out.println("Inside test one method");}
    public static void main (String [] args)
    {
        Combo7 ref = new Combo7();
        ref.m1();
    }
    void m1()
    {
        //dierctly
        System.out.println(x);
        test1();
        //object creation
        Combo7 c= new Combo7();
        System.out.println(c.x);
        c.test1();
    }
}
```



C:\Windows\System32\cmd.e

```
hods\Combo>java Combo7
a
Inside test one method
a
Inside test one method
```

```
//non static member
//non static area
//multiple classes
class Cyber
{
    char x='a';
    void test1()
    {
        System.out.println("Inside non static method.");}
}
class Combo8
{
    public static void main (String [] args)
    {
        Combo8 ref = new Combo8();
        ref.m1();
    }
    void m1()
    {
        //object creation
        Cyber c= new Cyber();
        System.out.println(c.x);
        c.test1();
    }
}
```

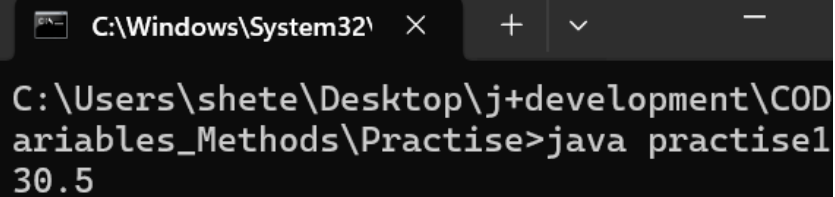


C:\Windows\System32\cmd.e

```
hods\Combo>java Combo8
a
Inside non static method.
```

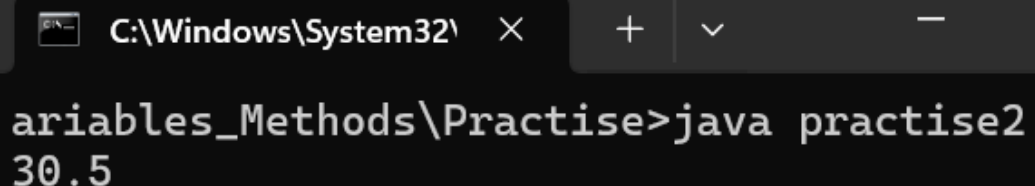
4. VARIABLES METHODS

```
1 class practise1{
2     static int var1=10;
3     float var2=20.5f;
4
5     public static void main(String [] args){
6         method1();
7     }
8
9
10 static void method1(){
11     practise1 p= new practise1();
12
13     float r=var1+p.var2;
14     System.out.println(r);
15 }
16 }
```



C:\Windows\System32\ x + v -
C:\Users\shete\Desktop\j+development\COD
ariables_Methods\Practise>java practise1
30.5

```
1 class practise2
2 {
3     static int var1=10;
4     float var2=20.5f;
5
6     public static void main(String [] args)
7     {
8         practise2 p =new practise2();
9         System.out.println(p.method1());
10    }
11 float method1()
12 {
13
14     float r=var1+var2;
15     return r;
16 }
17 }
```



C:\Windows\System32\ x + v - □
ariables_Methods\Practise>java practise2
30.5

4. VARIABLES METHODS

```
1 class practise3{
2
3     float reslut1;
4     static float result2;
5
6     public static void main (String [] args)
7     {
8
9         System.out.println("Start");
10        int a=10;
11        float b=20.5f;
12        practise3 var= new practise3();
13
14        var.reslut1=method1(a,b);
15        var.method2(a,b);
16
17        float finalr= var.reslut1+result2;
18        System.out.println("finalr:"+finalr);
19        System.out.println("Stop");
20    }
21
22    static float method1 (int a, float b)
23    {
24        practise3 p= new practise3();
25        System.out.println("Inside m1 method");
26        p.reslut1=a*b;
27        System.out.println("result1:"+p.reslut1);
28        return p.reslut1;
29    }
30
31
32    void method2(int a , float b)
33    {
34        System.out.println("inside m2 method");
35        result2=a+b;
36        System.out.println("result2:"+result2);
37    }
38 }
```

C:\Windows\System32\

```
Start
Inside m1 method
result1:205.0
inside m2 method
result2:30.5
finalr:235.5
Stop
```

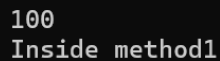
//3,4

4. VARIABLES METHODS

Non-Primitive or Reference Type

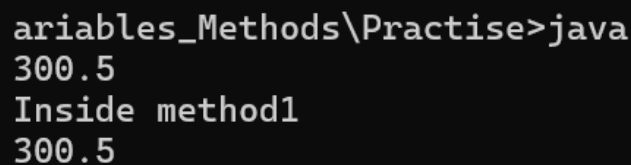
- Primitive data types are meant to store data which can be in different data formats.
- Non primitive variables will also store address but these addresses belongs to different classes.
- Non Primitive variables are also called as class type variables.

```
1 class Practice5
2 {
3     int a=100;
4     public static void main(String []args)
5     {
6         //local reference variable
7         Practice5 ref=new Practice5();
8         System.out.println(ref.a);
9         ref.method1();
10    }
11    void method1()
12    {
13        System.out.println("Inside method1");
14    }
15 }
```



```
100
Inside method1
```

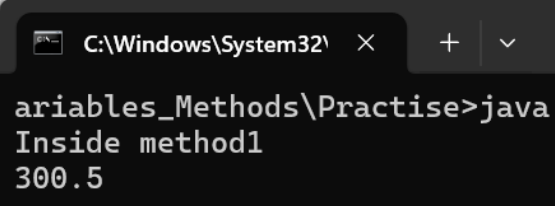
```
1 class Practice6
2 {
3     int a=100;
4     float b=200.5f;
5     public static void main(String []args)
6     {
7         //local reference variable
8         Practice6 ref1=new Practice6();
9         System.out.println(ref1.a+ref1.b);
10        method1(ref1); //Pass by reference
11    }
12    static void method1(Practice6 ref2)
13    {
14        System.out.println("Inside method1");
15        System.out.println(ref2.a+ref2.b);
16    }
17 }
```



```
variables_Methods\Practise>java
300.5
Inside method1
300.5
```


4. VARIABLES METHODS

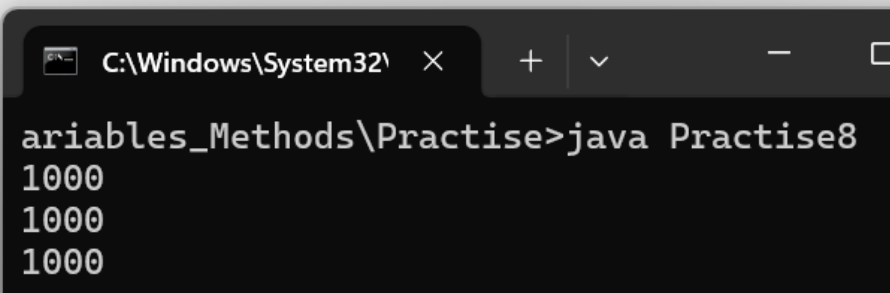
```
1 class Practice7
2 {
3     int a=100;
4     float b=200.5f;
5     public static void main(String []args)
6     {
7         Practice7 ref=new Practice7();
8         method1();
9         //System.out.println(ref.a+ref.b);
10    }
11    static Practice7 method1()
12    {
13        System.out.println("Inside method1");
14        Practice7 ref=new Practice7();
15        System.out.println(ref.a+ref.b);
16        return ref;
17    }
18 }
```



C:\Windows\System32\ x + v

ariables_Methods\Practise>java
Inside method1
300.5

```
1 class Practise8
2 {
3     int x=1000;
4     static Practise8 ref=new Practise8();
5     public static void main(String []args)
6     {
7         //Directly
8         System.out.println(ref.x);
9         //Classname
10        System.out.println(Practise8.ref.x);
11        //object
12        Practise8 refvar=new Practise8();
13        System.out.println(refvar.ref.x);
14    }
15 }
```

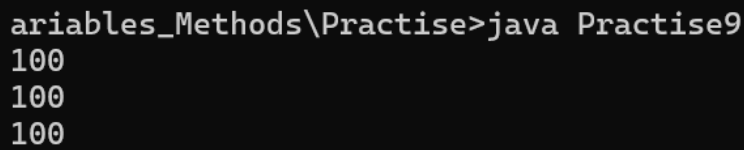


C:\Windows\System32\ x + v - □

ariables_Methods\Practise>java Practise8
1000
1000
1000

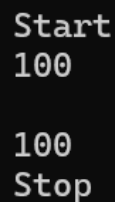
4. VARIABLES METHODS

```
1 class Practise9
2 {
3     int x=100;
4     static Practise9 refvar;
5     public static void main(String []args)
6     {
7         refvar=new Practise9();
8         //Directly
9         System.out.println(refvar.x);
10        //Classname
11        System.out.println(Practise9.refvar.x);
12        //object
13        Practise9 retval=new Practise9();
14        System.out.println(retval.refvar.x);
15    }
16 }
```



```
C:\Windows\System32\ > java Practise9
100
100
100
```

```
1 class Practise10
2 {
3     int x=100;
4     static Practise10 refvar=new Practise10();
5     public static void main (String []args)
6     {
7         System.out.println("start");
8         refvar.m2();
9         System.out.println();
10        refvar.m1();
11        System.out.println("stop");
12    }
13    static void m1(){
14        System.out.println(refvar.x);
15    }
16    void m2()
17    {
18        System.out.println(this.x);
19    }
20 }
```

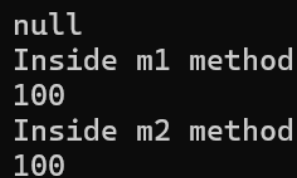


```
C:\Windows\System32\ > java Practise10
Start
100

100
Stop
```

4. VARIABLES METHODS

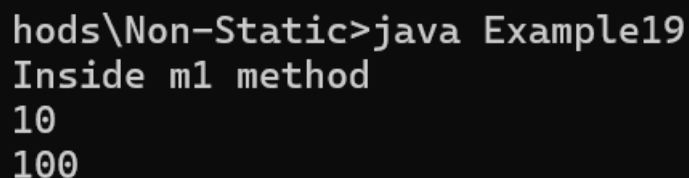
```
1 class Practise11
2 {
3     int x=100;
4     static Practise11 refvar;
5     public static void main(String []args)
6     {
7         System.out.println(refvar);
8         refvar=new Practise11();
9         m1();
10        refvar.m2();
11    }
12    static void m1()
13    {
14        System.out.println("Inside m1 method");
15        System.out.println(refvar.x);
16    }
17    void m2()
18    {
19        System.out.println("Inside m2 method");
20        System.out.println(refvar.x);
21    }
22 }
```



```

null
Inside m1 method
100
Inside m2 method
100
```

```
1 class Example19
2 {
3     int a=100;
4     void m1(int a)
5     {
6         System.out.println("Inside m1 method");
7         System.out.println(a);
8         System.out.println(this.a);
9     }
10    public static void main(String []args)
11    {
12        Example19 refvar=new Example19();
13        refvar.m1(10);
14    }
15 }
```

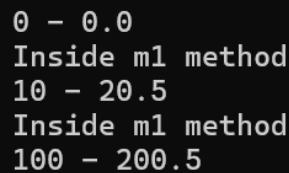


```

hods\Non-Static>java Example19
Inside m1 method
10
100
```

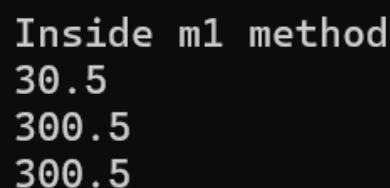
4. VARIABLES METHODS

```
1 class Example20
2 {
3     int a;
4     float b;
5     void m1(int a, float b)
6     {
7         System.out.println("Inside m1 method");
8         this.a=a;
9         this.b=b;
10    }
11    public static void main(String []args)
12    {
13        Example20 refvar=new Example20();
14        System.out.println(refvar.a+" - "+refvar.b);
15        refvar.m1(10,20.5f);
16        System.out.println(refvar.a+" - "+refvar.b);
17        Example20 ref=new Example20();
18        ref.m1(100,200.5f);
19        System.out.println(ref.a+" - "+ref.b);
20    }
21 }
```



```
0 - 0.0
Inside m1 method
10 - 20.5
Inside m1 method
100 - 200.5
```

```
1 class Example18
2 {
3     int a=100;
4     float b=200.5f;
5     void m1(int a, float b)
6     {
7         System.out.println("Inside m1 method");
8         System.out.println(a+b);
9         //1 Aproach
10        Example18 refvar=new Example18();
11        System.out.println(refvar.a+refvar.b);
12        //2 Aproach-this keyword
13        System.out.println(this.a+this.b);
14    }
15    public static void main(String []args)
16    {
17        Example18 refvar=new Example18();
18        refvar.m1(10,20.5f);
19    }
20 }
```



```
Inside m1 method
30.5
300.5
300.5
```

4. VARIABLES METHODS

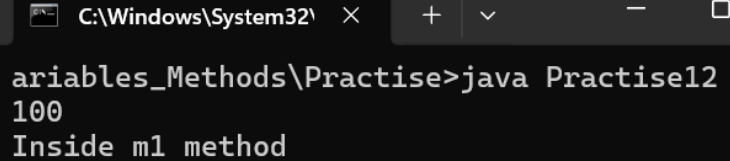
When to use this keyword

To differentiate between local and instance variables / non static variables when the variable names are same then we use this keyword.

//example21

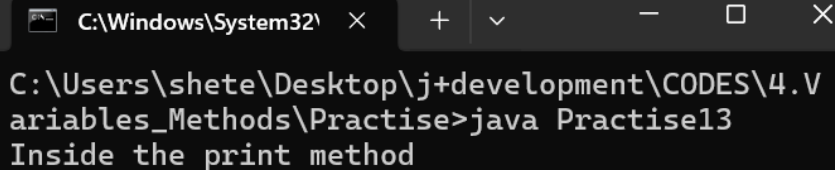
22

```
1  class Demo
2  {
3      static Sample refvar;
4  }
5  class Sample
6  {
7      int a=100;
8      static void m1()
9      {
10         System.out.println("Inside m1 method");
11     }
12 }
13 class Practise12
14 {
15     public static void main(String []args)
16     {
17         Demo.refvar=new Sample();
18         System.out.println(Demo.refvar.a);
19         Demo.refvar.m1();
20     }
21 }
```



```
C:\Windows\System32\ x + - □
variables_Methods\Practise>java Practise12
100
Inside m1 method
```

```
1  class System1
2  {
3      static System1 out1;
4      void println1(String Str)
5      {
6          System.out.println(Str);
7      }
8  }
9  class Practise13
10 {
11     public static void main(String []args)
12     {
13         System1.out1=new System1();
14         System1.out1.println1("Inside the print method");
15     }
16 }
```



```
C:\Windows\System32\ x + - □ x
C:\Users\shete\Desktop\j+development\CODES\4.V
variables_Methods\Practise>java Practise13
Inside the print method
```

//example23-25

4. VARIABLES METHODS

There are two ways to create an object

1. Name the object creation
2. Name + object creation

From a non-static context, we can access both static and non-static members.

But from a static context we can access only static members.

28

When to make members static?

Whenever the data or service which is **common** in such a situation we make the members **static**.

Whenever the data and service are **unique**, we make the member as **non-static**.

//cybersuccess

4. VARIABLES METHODS

IMP QUESTIONS

1. What are the local variable?

Variables which are declared within a method or constructor or blocks such variables are called as local variables.

2. Is it possible to print local variable without initializing? Why?

No, Local variables must be explicitly initialized because they are stored in the stack and Java does not assign them default values.

3. What are the static variables?

Variables which are declared outside the method or constructor or block but within the class and is prefix with static keyword such keyword called as static variables.

4. How can we directly access static members?

Static members are loaded by the JVM's ClassLoader at the time of class loading, before any object of the class is created. The ClassLoader loads the class into memory, and during the class initialization phase, all static variables and static blocks are executed once.

5. Why we can't use static keyword for local variables?

We cannot use the static keyword for local variables because static members belong to the class, whereas local variables belong to a method or block and exist only during method execution.

- static requires class-level storage
 - Local variables **do not exist at class loading time**
 - JVM cannot associate a local variable with the class
-

6. What are methods? Example

Methods are blocks of code defined inside a class that perform a specific task. Example: If my class is car then clutch(), brake(), acceleator() are my methods.

4. VARIABLES METHODS

7. When should we go for pass by value?

When we have to access local variables inside the methods or constructors also We go for **pass by value** when we want to **protect the original data** from being modified. In pass by value, a **copy of the variable** is passed to the method, so changes made inside the method **do not affect the original variable**

8. What is void?

void is a return type in Java that indicates a method does not return any value. It is used when a method only performs an action and does not need to send data back to the caller.

9. What type of memory does jvm consume why?

JVM consumes **RAM (main memory)** because Java programs are executed at runtime.

10. Why is the main method named as static?

Whenever JVM enters the stack the first thing what the JVM does it will go to the static pool and look for main if the main is present then it will start the execution if the main is not present then execution can't be started.

11. What are the non-static variables?

- Non-Static variables are the variables which are declared inside class and outside the methods, constructor and blocks.
 - They are **instance variables** that belong to an **object**, not to the class. Each object gets its **own separate copy**, and they are accessed using an **object reference**.
-

12. Is it possible to print static and non-static variables without initializing?

Yes

13. Who assigns default values for static and non-static variables;

- Static variables: JVM(Just-in-Time compiler) initializes while class loading
 - Non-Static variables: JVM initialize while object creation.
-

4. VARIABLES METHODS

14. What is the role of new in object creation.

The new keyword is meant to perform three actions.

- It is made to create or allocated memory
- It is responsible to load non static members(methods,variables).
- It is made to help generate an address along with JVM.

15. A single object creation will be responsible to load all not static members of a respective class within the object, HOW?

When an object is created using the new keyword, the JVM allocates heap memory for that object. During this process, the JVM loads all non-static (instance) variables and instance blocks into the object and assigns them default values, followed by constructor initialization.

16. When to make members static, non-static or local?

- **Static members:** Whenever the data or service which is **common** in such a situation we make the members static.
- **Non-Static members:** Use when data or behaviour **varies** from object to object.
- **Local:** Use when data is **temporary**, required only inside a method or block, and not needed outside.

EXAMPLE

Static variable: bankName → Same for all accounts (e.g., SBI)

Non-static variable: accountNumber, balance → Different for each customer

Local variable: withdrawAmount → Used only inside withdraw() method for calculation

Static → common data, Non-static → object-specific data, Local → temporary data inside method.

4 So who initializes what?		
Member Type	Who initializes?	When?
Static variables	JVM	Class loading
Instance variables	JVM (default)	Object creation
Instance variables (custom value)	Constructor	After default init
Static custom values	Static block	Class initialization

4. VARIABLES METHODS

When to use this keyword

To differentiate between local and instance variables / non static variables when the variable names are same then we use this keyword.